Build a fine-tuned AI support agent: from zero to a sellable product

I'd build this as a small SaaS product for small software companies that receive repetitive customer support questions. The first version should answer questions from their documentation, classify incoming tickets, draft replies, and escalate uncertain cases to a human.

The goal isn't just to fine-tune a model. It's to build something a business can actually use, while keeping training costs at ₹0 using Kaggle and your MacBook M5.

> **Product direction**
> A lightweight AI support inbox, not another general-purpose chatbot.

1. Define exactly what you're building

Give the project a temporary name: **SupportPilot**. You can rename it after validating demand.

A company signs up, creates a workspace, uploads its help documents, and gets a support assistant that can:

- Answer questions using the company's own documentation, with citations.
- Classify incoming messages into categories such as billing, technical issue, account access, and feature request.
- Draft replies in a consistent tone.
- Detect when information is missing or the model shouldn't answer.
- Escalate difficult cases to a human.
- Let the company review conversations and correct mistakes.

The first version will draft and recommend, not automatically send emails, issue refunds, or change customer accounts. That keeps the initial product simpler and safer.

What makes it a business?

A company isn't going to pay merely because you fine-tuned a model. It may pay if your product reduces the time staff spend answering repetitive questions without increasing incorrect answers.

Your initial target should be small SaaS companies with a modest support volume and a founder or small team handling support themselves. Avoid enterprise customers initially because their security, integration, and procurement requirements are heavier.

2. Choose the model and training method

Don't start by training a model from scratch. You'd spend your compute budget learning basic language patterns instead of solving your target problem.

> **Primary model: Qwen2.5-1.5B-Instruct**
> 1.5 billion parameters · Apache-2.0 license. Use this as your initial base model. It is small enough to experiment with on a free GPU, and the model is available on Hugging Face. Check the license and applicable terms again before commercial release.

> **Fallback: Qwen2.5-0.5B-Instruct**
> If the larger model runs out of memory or training becomes too slow, use this smaller model for initial experiments. Do not assume its output quality will be sufficient for customers.

What does fine-tuning actually teach it?

Use **LoRA (Low-Rank Adaptation)**. It trains a small set of additional parameters while leaving the original model weights frozen, reducing the cost of adaptation. Hugging Face's PEFT and TRL libraries support this workflow.

Your system will have three separate components:

Component	Role
Company knowledge base	Documentation, FAQs, product policies
RAG retrieval	Find relevant passages and provide sources
Fine-tuned Qwen model	Classify ticket, draft response, flag escalation
Support inbox	Suggested answer, evidence, review or escalation

This separation matters. RAG supplies current company facts; fine-tuning teaches the model how to handle support tasks. You won't have to retrain whenever a company updates its pricing or documentation.

3. Set up your development environment

Use your MacBook for application development, datasets, evaluation and initial inference. Use Kaggle for GPU training. Kaggle provides free GPU notebook sessions, but availability and weekly quotas vary.

Step 3.1: Create the project

Open Terminal on your Mac and run:

```
mkdir supportpilot
cd supportpilot

mkdir -p data notebooks models src
python3 -m venv .venv
source .venv/bin/activate

python -m pip install --upgrade pip
pip install streamlit pypdf sentence-transformers faiss-cpu \
    transformers torch accelerate datasets peft trl
```

If installation of a particular training dependency fails on macOS, don't waste time fixing the entire training stack locally. Train on Kaggle instead and use your Mac for the app.

Create this initial structure:

```
supportpilot/
├── app.py
├── requirements.txt
├── data/
│   ├── train.jsonl
│   ├── eval.jsonl
│   └── demo_docs/
├── notebooks/
│   └── finetune_support.ipynb
├── models/
│   └── supportpilot-lora/
└── src/
    ├── knowledge_base.py
    ├── assistant.py
    └── evaluation.py
```

Add the installed dependencies to requirements.txt when you're ready to deploy. Keep your training notebook's dependencies separate from the production app's dependencies.

Step 3.2: Confirm the hardware

Run:

```
python -c "import torch; print(torch.__version__); print('MPS:', torch.backends.mps.is_available())"
```

MPS: True means PyTorch can use Apple's Metal acceleration on your Mac. It doesn't guarantee that every model or training operation will fit in memory.

For your first training run, use Kaggle. This avoids depending on the exact memory configuration of your MacBook.

4. Build your training dataset

This is one of the most important parts of the project. Don't start by downloading a random support dataset and fine-tuning on it without defining what your product should do.

Your model should return a predictable structure for every customer message:

```
{
  "category": "billing",
  "priority": "medium",
  "needs_human": false,
  "draft_reply": "You can update your billing details in Settings > Billing.",
  "reason": "The provided documentation covers this question."
}
```

The application should validate that structure and display it in the dashboard.

Step 4.1: Define your categories

Start with these six:

Category	Example
billing	"Why was I charged twice?"
technical_issue	"The dashboard won't load."
account_access	"I can't reset my password."
feature_request	"Can you add CSV export?"
how_to	"How do I invite a teammate?"
other	Questions that don't fit the above

Priority should be **low**, **medium**, or **high**. Escalation should be a boolean.

Create examples for routine questions, missing information, frustrated customers, ambiguous requests, and questions that must be escalated. Don't train the model to invent refunds, account changes, or product policies.

Step 4.2: Start with a small, clean dataset

Aim for 300 to 500 high-quality examples for the first experiment, then expand toward 1,000 or more if your evaluation shows a benefit. These are practical starting targets, not guarantees of performance.

Create **data/train.jsonl**. Each line must be a separate JSON object:

```
{"messages":[{"role":"system","content":"You are a customer support triage assistant. Return only valid JSON
  with category, priority, needs_human, draft_reply, and reason. Never invent company policies. If evidence
  is missing, escalate."},{"role":"user","content":"Customer message: How do I invite my teammate?\nRelevant
  documentation: Workspace admins can invite teammates from Settings > Members by selecting Invite member."}
  ,{"role":"assistant","content":"{\"category\":\"how_to\",\"priority\":\"low\",\"needs_human\":false,\"draf
  t_reply\":\"A workspace admin can invite a teammate from Settings > Members by selecting Invite
  member.\",\"reason\":\"The documentation directly explains this process.\"}"}]}

{"messages":[{"role":"system","content":"You are a customer support triage assistant. Return only valid JSON
  with category, priority, needs_human, draft_reply, and reason. Never invent company policies. If evidence
  is missing, escalate."},{"role":"user","content":"Customer message: I was charged twice this month. Can
  you refund one payment?\nRelevant documentation: Billing disputes require review by the billing team. The
  assistant cannot issue refunds."},{"role":"assistant","content":"{\"category\":\"billing\",\"priority\":\"
  medium\",\"needs_human\":true,\"draft_reply\":\"I'm sorry about the duplicate charge. Our billing team
  needs to review the payments before confirming whether a refund is appropriate.\",\"reason\":\"The refund
  request requires human review.\"}"}]}

{"messages":[{"role":"system","content":"You are a customer support triage assistant. Return only valid JSON
  with category, priority, needs_human, draft_reply, and reason. Never invent company policies. If evidence
  is missing, escalate."},{"role":"user","content":"Customer message: Can I export my data?\nRelevant
  documentation: No information about data export is available."},{"role":"assistant","content":"{\"category
  \":\"feature_request\",\"priority\":\"low\",\"needs_human\":true,\"draft_reply\":\"I don't have enough
  verified information to confirm whether data export is supported. I'll flag this for the support team to
  clarify.\",\"reason\":\"The available documentation does not answer the question.\"}"}]}
```


These are illustrative examples. In a real dataset, vary the wording, product context, categories, and outcomes instead of duplicating templates with superficial changes.

Important dataset rules:

- Use fictional company policies for your initial demo.
- Do not train on private customer conversations without proper permission and appropriate data handling.
- Split the dataset by underlying scenario, not merely by random rows. Similar paraphrases in training and evaluation can make results look much better than they really are.
- Keep a separate eval.jsonl that is never used during training.

5. Fine-tune the model on Kaggle

We'll use supervised fine-tuning (SFT) with LoRA. This is the simplest useful training approach for teaching the model to return support classifications and structured draft responses. TRL's SFTTrainer supports conversational training data and PEFT adapters.

Step 5.1: Create your Kaggle notebook

1. Open Kaggle Notebooks and create a new notebook.
2. In notebook settings, select an available GPU accelerator.
3. Check that the GPU is working (code below).
4. Upload train.jsonl and eval.jsonl as a Kaggle dataset, or attach them to the notebook.

```
import torch
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print(torch.cuda.get_device_name(0))
```

Step 5.2: Install training dependencies

Run this in a notebook cell:

```
!pip install -q -U transformers datasets peft trl accelerate
```

If the notebook already has a compatible set of packages, avoid upgrading everything unnecessarily. Restart the session if the installed libraries require it.

Step 5.3: Load the dataset and base model

```
import torch
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import LoraConfig
from trl import SFTConfig, SFTTrainer

MODEL_ID = "Qwen/Qwen2.5-1.5B-Instruct"

files = {
    "train": "/kaggle/input/YOUR-DATASET/train.jsonl",
    "eval": "/kaggle/input/YOUR-DATASET/eval.jsonl",
}
dataset = load_dataset("json", data_files=files)

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(
    MODEL_ID,
    torch_dtype=torch.float16,
)
model.config.use_cache = False
```

Replace YOUR-DATASET with the actual Kaggle dataset folder name.

If you encounter an out-of-memory error, switch to the 0.5B model and reduce the sequence length and batch size. If the model loads successfully, continue with the 1.5B version.

Step 5.4: Configure LoRA

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    lora_dropout=0.05,
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[
        "q_proj",
        "k_proj",
        "v_proj",
        "o_proj",
        "gate_proj",
        "up_proj",
        "down_proj",
    ],
)
```

These settings are a reasonable starting point, not an optimized configuration. LoRA trains adapter weights rather than all the original model parameters.

Step 5.5: Format the conversational examples

Add this helper:

```
def format_example(example):
    return {
        "text": tokenizer.apply_chat_template(
            example["messages"],
            tokenize=False,
            add_generation_prompt=False,
        )
    }

train_ds = dataset["train"].map(format_example)
eval_ds = dataset["eval"].map(format_example)
```

This converts each conversation into the model's expected chat format.

Step 5.6: Train

```
training_args = SFTConfig(
    output_dir="/kaggle/working/supportpilot-checkpoints",
    num_train_epochs=2,
    per_device_train_batch_size=1,
    per_device_eval_batch_size=1,
    gradient_accumulation_steps=8,
    learning_rate=1e-4,
    fp16=True,
    gradient_checkpointing=True,
    max_length=768,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    save_total_limit=2,
    report_to="none",
    dataset_text_field="text",
)

trainer = SFTTrainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=eval_ds,
    processing_class=tokenizer,
    peft_config=lora_config,
)

trainer.train()
```

If your installed TRL version rejects an argument, check its version and the current SFTTrainer documentation rather than blindly changing multiple parameters.

Monitor training loss and evaluation loss. Falling training loss alone does not prove that the model has learned to handle unseen customer queries.

Step 5.7: Save the adapter

```
OUTPUT_DIR = "/kaggle/working/supportpilot-lora"
trainer.model.save_pretrained(OUTPUT_DIR)
tokenizer.save_pretrained(OUTPUT_DIR)
```

Download the supportpilot-lora folder from the Kaggle notebook's output files. Keep the base model separate. The adapter is not a complete model by itself.

You can also upload the adapter to a private or public Hugging Face repository. Keep in mind that a public adapter trained on sensitive customer data can leak information, so only publish adapters trained on data you are permitted to release.

6. Evaluate the model before building around it

Do not judge the fine-tuned model from three examples that look good.

Create at least 100 held-out test cases initially. Include routine questions, missing documentation, requests for refunds, ambiguous messages, and adversarial instructions embedded in customer messages.

Compare two systems on exactly the same test cases:

- **Baseline:** The original Qwen model with a well-designed prompt.
- **Fine-tuned:** Qwen with your LoRA adapter and the same prompt, retrieval results, and output requirements.

Track these metrics:

Metric	What it tells you
Category macro-F1	Whether ticket classification works across categories

Metric	What it tells you
Valid JSON rate	Whether the model produces parseable output
Escalation recall	Whether it catches cases that need human review
Unsupported-claim rate	How often it invents facts not supported by the evidence
Human-rated reply quality	Whether the draft is useful and appropriate
Median response time	Whether the product feels responsive

For escalation, prioritize recall on genuinely risky cases, even if this means sending some answerable cases to a human. Choose thresholds based on observed evaluation results, not an arbitrary confidence score generated by the model.

If fine-tuning doesn't improve the baseline, don't force it into the product. Better retrieval, clearer prompts, or a different model may solve the problem more effectively.

7. Build the knowledge base (RAG)

Now the product needs to answer questions using a company's own documentation. This is a separate component from fine-tuning.

For your MVP, support only PDF, TXT, and Markdown uploads. Skip crawling entire websites and integrating ticketing platforms until the basic workflow works.

Step 7.1: Install the libraries

On your Mac:

```
pip install pypdf sentence-transformers faiss-cpu
```

Use sentence-transformers/all-MiniLM-L6-v2 to create embeddings and FAISS to search for relevant text chunks. Both are suitable for a small local prototype.

Step 7.2: Create src/knowledge_base.py

```
from pathlib import Path
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
import faiss
import numpy as np

embedder = SentenceTransformer(
    "sentence-transformers/all-MiniLM-L6-v2"
)

def read_document(filename, content):
    suffix = Path(filename).suffix.lower()
    if suffix == ".pdf":
        import io
        reader = PdfReader(io.BytesIO(content))
        return "\n".join(page.extract_text() or "" for page in reader.pages)
    if suffix in {".txt", ".md"}:
        return content.decode("utf-8", errors="replace")
    raise ValueError("Only PDF, TXT, and Markdown are supported.")

def chunk_text(text, size=120, overlap=20):
    words = text.split()
    chunks = []
    for start in range(0, len(words), size - overlap):
        chunk = " ".join(words[start:start + size])
        if chunk.strip():
            chunks.append(chunk)
    return chunks

def build_index(documents):
    # documents: list of {"filename": str, "text": str}
    chunks = []
    for doc in documents:
        for i, chunk in enumerate(chunk_text(doc["text"])):
            chunks.append({
                "text": chunk,
                "source": doc["filename"],
                "chunk": i,
            })
    if not chunks:
        raise ValueError("No readable text found in uploaded documents.")
    vectors = embedder.encode(
        [c["text"] for c in chunks],
        normalize_embeddings=True,
        convert_to_numpy=True,
    ).astype("float32")
    index = faiss.IndexFlatIP(vectors.shape[1])
    index.add(vectors)
    return index, chunks

def retrieve(question, index, chunks, top_k=4):
    query_vector = embedder.encode(
        [question],
        normalize_embeddings=True,
        convert_to_numpy=True,
    ).astype("float32")
    scores, ids = index.search(
        query_vector,
        min(top_k, len(chunks)),
    )
    results = []
    for score, idx in zip(scores[0], ids[0]):
        if idx >= 0:
            results.append({
                **chunks[idx],
                "score": float(score),
            })
    return results
```

This is a basic retrieval implementation. The similarity scores are ranking signals, not calibrated probabilities that a passage is correct.

Step 7.3: How it works

When a company uploads its help documentation:

1. Extract the text.
2. Split it into small overlapping chunks.
3. Convert each chunk into an embedding.
4. Store the embeddings in a searchable index.
5. When a customer asks a question, retrieve the most relevant passages.
6. Give those passages to the fine-tuned model as evidence.

The product should display the document name beside each draft so the support operator can verify the answer. If no useful evidence is found, the system should abstain or escalate rather than fabricate a policy.

8. Connect the fine-tuned model to the knowledge base

Create **src/assistant.py**.

This file takes the retrieved passages, sends them to the fine-tuned model, and converts the response into a structure your application can display.

```
import json
import re
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel

BASE_MODEL = "Qwen/Qwen2.5-1.5B-Instruct"
ADAPTER_PATH = "models/supportpilot-lora"

def load_assistant():
    tokenizer = AutoTokenizer.from_pretrained(BASE_MODEL)
    model = AutoModelForCausalLM.from_pretrained(
        BASE_MODEL,
        torch_dtype=torch.float16,
    )
    model = PeftModel.from_pretrained(model, ADAPTER_PATH)
    device = (
        "mps" if torch.backends.mps.is_available()
        else "cuda" if torch.cuda.is_available()
        else "cpu"
    )
    # ... (remainder of this file, including generate_reply and the
    #      JSON parsing/validation, is not included in the source document)
```

The validation here is deliberately basic. Before a real release, also validate field types and lengths, reject malformed or unexpected output, and test for prompt injection. Never treat a model-generated needs_human: false as proof that a response is safe.

Keep the model and adapter on disk locally while developing. Don't load the model afresh for every customer message.

9. Build the actual web application

Use Streamlit for the first UI. It will get you to a working demo faster than building a separate React frontend and backend.

Your initial interface should have:

- A sidebar for uploading documentation.
- A text box for a customer message.
- A ticket category and priority.
- An AI-generated draft reply.
- A list of source passages.
- A clear escalation status.

- A way for the operator to edit the reply before using it.

Create app.py

```
import streamlit as st

# ... (top of app.py -- imports, sidebar upload, get_model(), index building and
#      the form opening -- is not included in the source document)

            placeholder="Example: How do I invite a teammate?",
            height=130,
        )
        submitted = st.form_submit_button("Generate support draft")

    if submitted and question.strip():
        with st.spinner("Analyzing the request..."):
            tokenizer, model, device = get_model()
            passages = retrieve(question, index, chunks)
            result = generate_reply(
                question,
                passages,
                tokenizer,
                model,
                device,
            )

        st.subheader("Suggested response")
        st.write("Category:", result["category"])
        st.write("Priority:", result["priority"])

        if result["needs_human"]:
            st.warning("Human review required")
        else:
            st.info("Review the draft before sending")

        draft = st.text_area(
            "Edit the reply",
            value=result["draft_reply"],
            height=160,
        )
        st.write("Reason:", result["reason"])

        st.subheader("Supporting evidence")
        if passages:
            for passage in passages:
                with st.expander(passage["source"]):
                    st.write(passage["text"])
                    st.caption(
                        f"Retrieval similarity: {passage['score']:.3f}"
                    )
        else:
            st.warning("No relevant documents were retrieved.")

        st.caption(
            "Demo only: this app does not send replies or change accounts."
        )
else:
    st.info(
        "Upload a PDF, TXT, or Markdown help document to begin."
    )
```

Create an empty **src/__init__.py** file too, so Python can import your modules reliably.

Run the application:

```
streamlit run app.py
```

Open the local URL Streamlit prints in the terminal. Upload a fictional product FAQ and test at least 20 questions.

Expected result: the app retrieves relevant passages, classifies a customer message, generates a draft, and shows its supporting evidence.

This is a functional prototype, not yet a production-ready multi-company SaaS. It currently rebuilds its retrieval index on each app run, doesn't persist tickets, and has no authentication or tenant isolation.

10. Add the features that make this a real product

Once the local demo works, stop adding ML features for a moment. Build the features a paying customer would expect.

A. Persistent ticket inbox

Save each message, generated draft, category, priority, status, and timestamp. Let operators reopen previous tickets and edit drafts.

B. Company workspaces

Each company should have its own account, documentation, tickets, and settings. Users from one company must never be able to access another company's data.

C. Feedback and corrections

Add buttons for "Useful", "Incorrect", and "Needs escalation". Save corrections with explicit consent and without unnecessary personal data. Review them before adding examples to future training datasets.

D. A measurable value dashboard

Show tickets processed, drafts accepted, escalation rate, and operator-reported time saved. Don't claim time or cost savings until you have actual measurements.

Step 10.1: Add persistence and authentication

For the first hosted version, you can use Supabase for PostgreSQL, authentication, and document storage. Its free tier has quotas, so check current limits before relying on it for a commercial deployment.

Start with these tables:

Table	Important fields
workspaces	id, name, created_at
workspace_members	workspace_id, user_id, role
documents	id, workspace_id, filename, status
tickets	id, workspace_id, message, category, priority, status
drafts	id, ticket_id, body, needs_human, reviewed_by
feedback	id, ticket_id, rating, correction

Use UUIDs for IDs and foreign keys for relationships. Every ticket, document, draft, and feedback record must belong to a workspace.

Security requirement: implement and test PostgreSQL Row Level Security policies for every tenant-owned table. Never trust a workspace_id supplied by the browser without checking membership on the server. Supabase documents how authentication and RLS work together.

Also:

- Keep service-role keys and other secrets on the server, never in frontend code.
- Don't expose raw uploaded documents through public URLs.
- Add document deletion and ticket deletion.
- Avoid collecting personal information you don't need.

- Don't use customer conversations for model training by default.
- Keep a record of who reviewed or changed a draft.

Step 10.2: Add a review workflow

Every ticket should have one of three states:

- **Draft ready:** the AI has prepared a response for review.
- **Needs review:** the AI has flagged uncertainty or a sensitive action.
- **Reviewed:** a human has approved or edited the response.

Do not automatically send replies in the first release. The initial product should assist the support operator, not act as an unrestricted agent.

11. Deploy it without paying for hosting

This is the part where you need to distinguish a free demo from a production service.

Development and training

MacBook M5 + Kaggle. No paid GPU required for the planned small-model experiments.

Public web interface

Streamlit Community Cloud is a reasonable place to try hosting the interface. Check its current limits before deployment.

Model inference

A public web app also needs somewhere to run the model. Free CPU hosting can be slow or memory-limited, and free GPU availability isn't guaranteed. A hosted inference API may have quotas or charges. Don't assume a permanently free, reliable production endpoint exists.

Your deployment sequence

1. Get streamlit run app.py working locally.
2. Upload the source code to a GitHub repository. Keep model weights and secrets out of Git.
3. Deploy the UI to Streamlit Community Cloud for a public demo.
4. Test whether the selected hosting environment can run the model and adapter within its resource limits. If it cannot, keep inference local for the initial demonstration or evaluate a suitable hosted inference option.
5. Add Supabase only after the core workflow is reliable.
6. Never put confidential customer data into a public demo.

For the first version, a public demo with fictional documents and a clear disclaimer is enough to approach potential customers. You don't need to solve scalable inference, billing, and enterprise security before validating demand.

12. A four-week execution plan

Follow this order. Don't try to build everything simultaneously.

Week 1 — Dataset and baseline

- Create the repository and Kaggle notebook.
- Write 300 to 500 training examples and a separate evaluation set.
- Run the original model on test cases.
- Record classification, JSON validity, and escalation performance.

Deliverable: dataset, baseline results, working training setup.

Week 2 — Fine-tuning and retrieval

- Fine-tune Qwen with LoRA on Kaggle.
- Download and load the adapter on your Mac.
- Build the document ingestion and retrieval pipeline.
- Compare the original and fine-tuned models on identical test cases.

Deliverable: reproducible training run and measurable evaluation.

Week 3 — Product MVP

- Build the Streamlit interface.
- Show editable drafts, classifications, escalation status, and citations.
- Test failure cases and malformed outputs.
- Record a short product demo.

Deliverable: a working end-to-end local application.

Week 4 — Validation and first pilot

- Publish a demo with fictional documents.
- Contact 10 to 20 small SaaS founders or support leads.
- Show them the product and ask about their existing workflow.
- Offer a limited pilot and measure whether it actually saves time.
- Prioritize the features requested by people who are willing to use it.

Deliverable: real user feedback and ideally one pilot customer.

These are target milestones, not guaranteed deadlines. If model setup or evaluation takes longer, postpone UI polish rather than skipping the evaluation.

13. How to get the first paying customer

Don't start by marketing this as a "fine-tuned LLM platform". Most potential customers won't care about LoRA.

Sell the outcome: less time spent answering repetitive support questions, with a human reviewing the responses.

A simple initial offer:

- Free, short trial using the customer's non-sensitive documentation.
- Support for one workspace and one knowledge base.
- AI-generated draft replies with evidence and escalation.
- A review meeting to compare results against the existing process.
- A paid monthly plan only after the workflow proves useful.

Don't choose a permanent price before speaking with customers. First learn their support volume, the cost of handling tickets, and what they'd pay to reduce that work.

14. Your final checklist

- ■ Create the GitHub repository and Kaggle notebook
- ■ Build and split the support training dataset
- ■ Measure the base-model baseline
- ■ Train and save the LoRA adapter
- ■ Compare the fine-tuned model against the baseline
- ■ Implement document ingestion and retrieval

- Build the Streamlit support inbox
- Test citations, escalation, and malformed outputs
- Deploy a public demo with fictional data
- Interview potential customers and seek a pilot

One final recommendation
Your first success criterion should be a working local product, not a trained model. If the fine-tuned version doesn't outperform the original model on your test set, keep the stronger baseline and continue building the product. That is the more commercially sensible decision.